

Containers

Announcements

List Review: Understanding []

```
>>> digits = [1, 8, 0, 1]
```

Make a new list by describing every element

```
>>> [d * 100 for d in digits if d < 5]  
[100, 0, 100]
```

Make a new list by telling Python how to create every element

```
>>> digits[1]  
8
```

Look up one element

```
>>> digits[100]  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
IndexError: list index out of range
```

```
>>> [d * 100 for d in digits if d < 5][1]  
0
```

```
>>> digits[1:]  
[8, 0, 1]
```

Make a new list with some of the elements

```
>>> same_digits = [digits[0]] + digits[1:]  
>>> same_digits  
[1, 8, 0, 1]
```

Create a new list with all of the elements in the first list followed by all of the elements in the second list

```
>>> digits[:1000]  
[1, 8, 0, 1]  
>>> digits[1000:]  
[]
```

Ok to go off the end!

Recursion Example: Reverse

```
def reverse(s):  
    """Return s in reverse order.  
  
    >>> reverse([4, 6, 2])  
    [2, 6, 4]  
    """  
    if not s:  
        return []  
    return reverse(s[1:]) + [s[0]]
```

(A) `reverse(s[1:] + [s[0]])`

(B) `[s[-1]] + reverse(s[:-1])`

(C) `[s[-1 * i] for i in range(len(s))]`

(D) `[s[-x + 1] for x in range(reverse(s))]`

What do each of these do?
Which correctly reverse?

pollev.com/c88c

Recursion Example: Reverse

```
def reverse(s):  
    """Return s in reverse order.  
  
    >>> reverse([4, 6, 2])  
    [2, 6, 4]  
    """  
    if not s:  
        return []  
    return reverse(s[1:]) + [s[0]]
```

~~(A)~~ `reverse(s[1:] + [s[0]])`

Runs forever

(B) `[s[-1]] + reverse(s[:-1])`

[2] [4, 6] [6, 4]

~~(C)~~ `[s[-1 * i] for i in range(len(s))]`

s[-1*0] s[-1*1] s[-1*2] 0, 1, 2
[4, 2, 6]

~~(D)~~ `[s[-x + 1] for x in range(reverse(s))]`

Runs forever

What do each of these do?
Which correctly reverse?

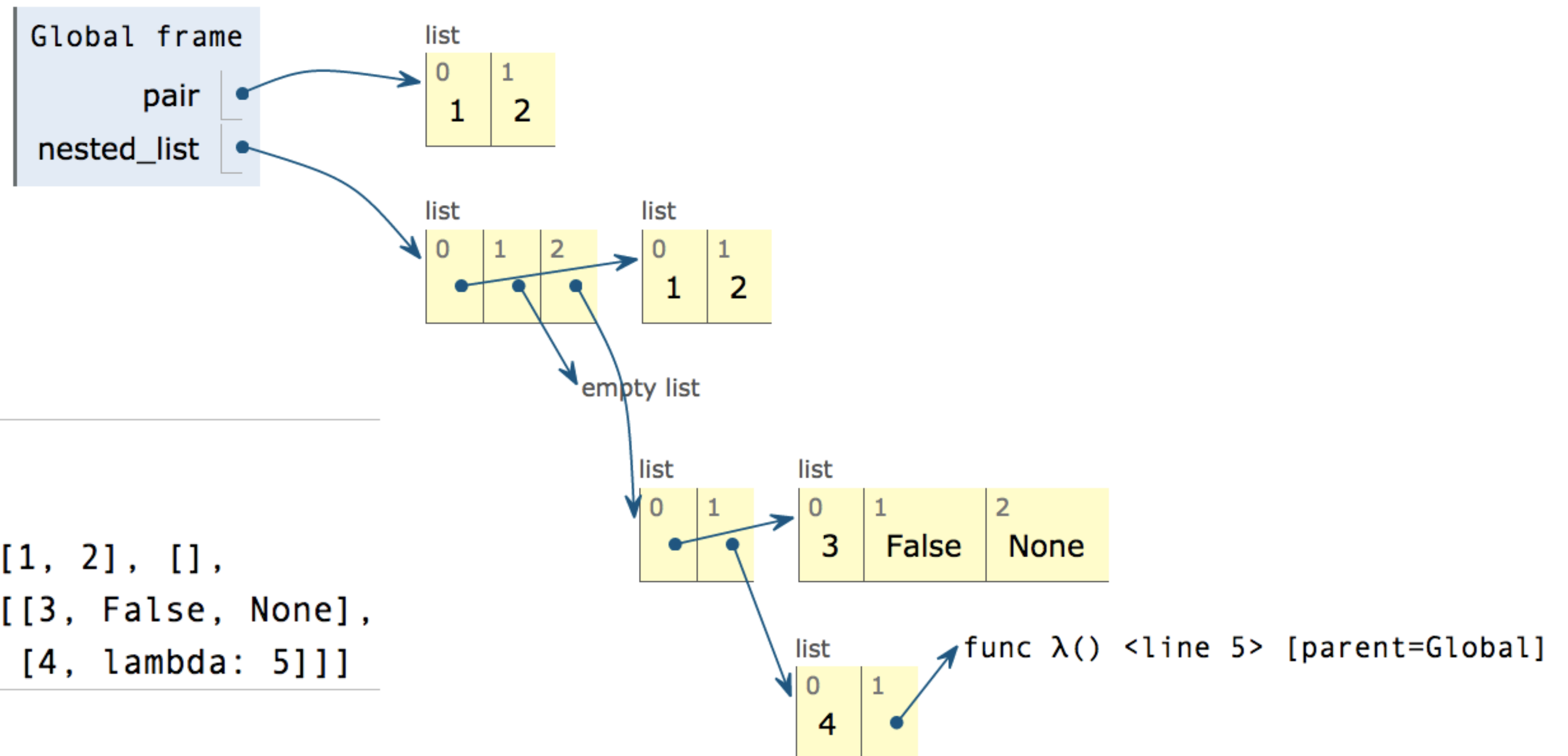
pollev.com/c88c

Box-and-Pointer Notation

Box-and-Pointer Notation in Environment Diagrams

Lists are represented as a row of index-labeled adjacent boxes, one per element

Each box either contains a primitive value or points to a compound value



```
1 pair = [1, 2]
2
3 nested_list = [[1, 2], [],
4                 [[3, False, None],
5                 [4, lambda: 5]]]
```

Discussion Question

What's the environment diagram? What gets printed?

```
def f(s):  
    x = s[0]  
    return [x]  
  
t = [3, [2+2, 5]]  
u = [f(t[1]), t]  
print(u)
```

pollev.com/c88c

Double-Eights with a List

Implement `double_eights`,
which takes a list `s` and returns whether two consecutive items are both 8.

using positions (indices)...

```
def double_eights(s):  
    """Return whether two consecutive items  
    of list s are 8.
```

```
>>> double_eights([1, 2, 8, 8])
True
>>> double_eights([8, 8, 0])
True
>>> double_eights([5, 3, 8, 8, 3, 5])
True
>>> double_eights([2, 8, 4, 6, 8, 2])
False
```

```

for i in range(len(s)-1):
    if s[i] == 8 and s[i+1] == 8:
        return True
return False

```

using slices...

```
def double_eights(s):  
    """Return whether two consecutive items  
    of list s are 8.
```

```
>>> double_eights([1, 2, 8, 8])
True
>>> double_eights([8, 8, 0])
True
>>> double_eights([5, 3, 8, 8, 3, 5])
True
>>> double_eights([2, 8, 4, 6, 8, 2])
False
```

```

if s[:2] == [8, 8]:
    return True
elif len(s) < 2:
    return False
else:
    return double_eights(s[1:])

```

Processing Container Values

Aggregation

Several built-in functions take iterable arguments and aggregate them into a value

- `sum(iterable[, start]) -> value`

Return the sum of an iterable (not of strings) plus the value of parameter 'start' (which defaults to 0). When the iterable is empty, return start.

- `max(iterable[, key=func]) -> value`
`max(a, b, c, ...[, key=func]) -> value`

With a single iterable argument, return its largest item.
With two or more arguments, return the largest argument.

- `all(iterable) -> bool`

Return True if `bool(x)` is True for all values `x` in the iterable.
If the iterable is empty, return True.

(Demo)

Summation

```
def cube(k):  
    return pow(k, 3)
```

```
def summation(n, term):  
    """Sum the first n terms of a sequence.
```

```
>>> summation(5, cube)
```

```
225  
"""
```

```
total, k = 0, 1
```

```
while k <= n:
```

```
    total, k = total + term(k), k + 1
```

```
return total
```

1 + 8 + 27 + 64 + 125

```
def summation2(n, term):
```

```
    return
```

pollev.com/c88c

Built-in aggregations:

- `sum(iterable[, start])` -> value

Return the sum of an iterable plus the value of parameter 'start' (which defaults to 0).

- `max(iterable[, key=func])` -> value
`max(a, b, c, ...[, key=func])` -> value

With a single iterable argument, return its largest item.

- `all(iterable)` -> bool

Return True if `bool(x)` is True for all values `x` in the iterable.

Summation

```
def cube(k):  
    return pow(k, 3)
```

```
def summation(n, term):  
    """Sum the first n terms of a sequence.
```

```
>>> summation(5, cube)
```

```
225  
"""
```

```
total, k = 0, 1
```

```
while k <= n:
```

```
    total, k = total + term(k), k + 1
```

```
return total
```

1 + 8 + 27 + 64 + 125

```
def summation2(n, term):  
    return sum( [term(x) for x in range(1, n + 1)] )
```

pollev.com/c88c

Built-in aggregations:

- `sum(iterable[, start])` -> value

Return the sum of an iterable plus the value of parameter 'start' (which defaults to 0).

- `max(iterable[, key=func])` -> value
`max(a, b, c, ...[, key=func])` -> value

With a single iterable argument, return its largest item.

- `all(iterable)` -> bool

Return True if `bool(x)` is True for all values `x` in the iterable.

Recursion Example: All Possible Sums

```
def sums(n: int) -> list[list[int]]:  
    """Return a list of all of the possible lists of  
    positive integers whose elements add up to n.
```

Write down an example!

```
>>> sums(3)  
[[1, 1, 1], [1, 2], [2, 1], [3]]  
"""
```

```
result = []
```

```
for first in range(1, n):
```

result: []

first: 1

```
    result = result +
```

```
return result + [[n]]
```

Make some return values
where 1 is first?

[[1, 1, 1], [1, 2]]

sums(2): [[1, 1], [2]]

What expression can we write
that results in [1, 1, 1], [1, 2]?

sums(2)

Recursion Example: All Possible Sums

```
def sums(n: int) -> list[list[int]]:
```

```
    """Return a list of all of the possible lists of
    positive integers whose elements add up to n.
```

Write down an example!

```
>>> sums(3)
[[1, 1, 1], [1, 2], [2, 1], [3]]
"""
```

```
result = []
```

result: []

```
for first in range(1, n):
```

first: 1

```
    result = result + [[first] + rest for rest in sums(n - first)]
```

```
return result + [[n]]
```

Make some return values
where 1 is first?

[[1, 1, 1], [1, 2]]

sums(2): [[1, 1], [2]]

What expression can we write [[1] + rest for rest in sums(2)]
that results in [1, 1, 1], [1, 2]?

Spring 2023 Midterm 2 Question

Definition. A *prefix sum* of a sequence of numbers is the sum of the first n elements for some positive length n .

(a) (4.0 points)

Implement `prefix`, which takes a list of numbers `s` and returns a list of the prefix sums of `s` in increasing order of the length of the prefix.

```
def prefix(s):  
    """Return a list of all prefix sums of list s.
```

```
>>> prefix([1, 2, 3, 0, 4, 5])
```

```
[1, 3, 6, 6, 10, 15]
```

```
>>> prefix([2, 2, 2, 0, -5, 5])
```

```
[2, 4, 6, 6, 1, 6]
```

```
"""
```

```
return [_____ for k in _____]
```

(a)

(b)

ii. (1.0 pt) Fill in blank (b).

☐ `s`

☐ `[s]`

☐ `s[1:]`

☐ `range(s)`

☐ `range(len(s))`

Spring 2023 Midterm 2 Question

Definition. A *prefix sum* of a sequence of numbers is the sum of the first n elements for some positive length n .

(a) (4.0 points)

Implement `prefix`, which takes a list of numbers `s` and returns a list of the prefix sums of `s` in increasing order of the length of the prefix.

```
def prefix(s):  
    """Return a list of all prefix sums of list s.
```

```
>>> prefix([1, 2, 3, 0, 4, 5])
```

```
[1, 3, 6, 6, 10, 15]
```

```
>>> prefix([2, 2, 2, 0, -5, 5])
```

```
[2, 4, 6, 6, 1, 6]
```

```
"""          sum(s[:k+1])          range(len(s))
```

```
return [_____ for k in _____]
```

(a)

(b)

ii. (1.0 pt) Fill in blank (b).

☐ `s`

☐ `[s]`

☐ `s[1:]`

☐ `range(s)`

☐ `range(len(s))`

Strings

'Demo'